Wie wurde die Evaluation der Software durchgeführt?	Hunderte Testläufe, in denen das Programm ausgeführt wurde und unterschiedliche Module isoliert und miteinander beobachtet wurden. Wenn Fehler aufgetreten sind, wurden sie ausgebessert, bis dieses Programm erreicht wurde.			
Wie wurde nicht getestet.	Es wurden keine Tests mit einer größeren Menge an Testpersonen durchgeführt. Aus den Studien ließen sich möglicherweise zusätzliche Probleme identifizieren.			
Was waren Dinge die für die Evaluation betrachtet wurden?	Es wurde das Timing, die korrekte Übergabe von Auswahlen, ein gleiches Verhalten überall im System, die Synchronität von Animationen und Motorbewegung sowie die intuitive Verständlichkeit für die Evaluation in den Testläufen betrachtet.			
Systeme und Module	Erklärung	Funktioniert	Probleme	mögliche Verbesserungen
System Standard				
analyze_logs	Analysedatei entnimmt die Stressbewertungen aus den JSON Dateien, die der Logger mit den Daten jeder Session füllt, also Stressbewertung vorher, nachher, Auswahl der Übung und Dauer. Die Werte werden korrekt aus den JSON Dateien entnommen.	Funktioniert		Logging Daten könnten noch komplexer miteinander in Verbindung gebracht werden
animation_cache	Läd Animationen, bevor das System startet.	Funktioniert	Funktioniert, aber verursacht das Problem, dass das Programm lange laden muss, bevor es beginnt. Es kann in der Main definiert werden, welcher Ordner geladen werden soll oder ob überhaupt.	Gesamtes Verhalten analysieren und konstante Reinigung und Cache überarbeiten?

audio_manager	Steuert die Anwendung des richtigen Audiopfades je nach ausgewählter Sprache. Funktioniert zuverlässig. Wendet außerdem die in der volume_ui gesetzte Lautstärke auf alle Auswahlen/UIs an.	Funktioniert		
custum_ui	Funktioniert wie von mir geplant. Werte werden korrekt übernommen, Audios werden korrekt gesetzt. Animationen werden korrekt ausgewählt. Alles funktioniert wie gewünscht: die Menge an Audios verursacht ein gewisses Delay beim Start der Übungen.	Funktioniert	Kann dazu kommen das der Übergang zwischen Audiodateien etwas unsauber ist.	
design_system	Wendet korrekt die definierten Farben über das gesamte System an. In der Config Datei sind Werte für Farben, Abstände etc. definiert.	Funktioniert		Dynamisches System einbauen, welches Fonts je nach Sprache auswählt und auf das System anwendet. Noch mehr Funktionen aus Auswahlen in das design_system verlagern, damit es zentraler wird?
duration_ui	Auswahl der Zeit funktioniert wie gewünscht. Die Auswahl der Zeit wird richtig auf die Übung angewendet (gemessen).	Funktioniert		
email_sender	Funktioniert wie gewünscht. Schickt mail. Kann in der Email nachgesehen werden Statistikdatei wird auftauchen.	Funktioniert	Google hat viele Sicherheitsrichtlinien was initiales Setup erschwert	Automatisches Verbinden mit öffentlichen Netzwerken beeinhaltet ganz eigene Probleme
evaluation_ui	Auswahl der Stresszustände sollte intuitiv verständlich sein. Man kann auch keine Angabe machen. Die jeweilige Auswahl wird korrekt gespeichert.	Funktioniert		
greetings_ui	Spielt Audios nacheinander mit definiertem Text ab. Funktioniert, wird aber nicht verwendet, da keine entsprechenden Audios existieren.	Funktioniert		Audios hinzufügen
intro	Spielt die Animation korrekt ab und wechselt wie gewünscht zurück in den Sleep Modus, wenn der Sensor keinen Menschen mehr wahrnimmt.	Funktioniert		Audio hinzufügen. Begrüßung?

language_manager	Funktioniert ausgesprochen gut. Mit der Auswahl der Sprache in der language_ui wird diese auch im language_manager gesetzt. Eine globale Instanz lang wird erstellt, die dafür sorgt, dass alle Auswahlen die korrekte JSON für die Anzeige der Sprache verwenden. Ein einfaches, unkompliziert erweiterbares System, welches erlaubt, Sprachen mit nur einer JSON Datei global für das System zu implementieren.	Funktioniert	Sprachen wie Mandarin, kyrillische Buchstaben etc. lassen sich nicht mit jedem Font schreiben. Ab einer bestimmten Flaggen Anzahl könnten Probleme entstehen. KI/Übersetzer machen Fehler, es kann sein, dass die Übersetzung inkorrekten Inhalt wiedergibt.	Dynamisches System einbauen, welches automatisch Font auswählt. Ansonsten sehe ich keine direkten Verbesserungen die notwendig wären
language_ui	Funktioniert ausgesprochen gut. Nach der Erstellung und Ablage der JSON Datei einer beliebigen Sprache wird diese sofort auch in der language_ui angezeigt. Es muss keine Veränderung in der language_ui vorgenommen werden. Alles funktioniert wie es sollte. Die jeweilige Flagge muss nur mit dem identischen Namen der JSON Datei im Flaggen-Ordner abgelegt werden, und sie wird automatisch der Sprache zugeordnet.	Funktioniert		Statt Scrolleffekt Knopf zum Anzeigen weiterer Sprachen einbauen
logger	Speichert die Übung, Auswahl der Dauer und das ausgewählte Stresssymbol. Wird in der Analysedatei miteinander kontextualisiert. Da eine zufällige ID für jede Sitzung erstellt wird, kann auch die Menge der Wiederholungen in der Analysedatei verwertet werden.	Funktioniert		

logic	Funktioniert ausgesprochen gut. Animationen werden hier gebaut und mit dem Motorcontroller zusammengeführt. Wenn eine neue Übung für das System erstellt werden soll, muss hier nur eine der bereits bestehenden Funktionen kopiert und angepasst werden. Also der Ordner der Animation, der Sound, der Name, der in der Main aufgerufen wird, und welche Motorfunktion verwendet werden soll. Die entsprechende Übung muss dann in der ui (Übungsauswahl) referenziert werden, damit sie als Auswahl auftaucht. Wenn dann in der Main das Verhalten ergänzt wird, ist die Übung schnell ausführbar. Schnell ergänzbar und funktionell. Dateien, die geöffnet werden müssen: logic, main, Motorfunktionen.	Funktioniert		
main	Kontrolliert den Verlauf der Übung. Gesamte Struktur die auch in den Abbildungen für die Systeme im Bericht dargestellt ist wird hier festgelegt. Alles wird hier kontrolliert und Zusammengebracht/ an die einzelnen Dateien geschickt wie beispielsweise die Navigation der Tasten die dann weiter an die derzeit geöffnete ui geschickt wird.	Funktioniert		

MotorC	Hier werden ankommende Befehle ausgelesen und ausgehende Befehle geschrieben. Die Motorbewegung wird hier als Befehl geschrieben. Der Befehl kommt von der Steuerung, die die entsprechenden Übungen aus MotorFunktionen in Übungsform bringt. Wenn ein Befehl der navigation hier ankommt, wird eine entsprechende Unterscheidung getroffen, welche Taste es ist und ob eine Auswahl getriggert wird oder nicht. Das wird dann weiter an die main geschickt, von wo es in den Auswahlen die Usereingabe steuert. Außerdem wird hier der Sensorwert für den Thermalsensor angenommen und entsprechend ein Callback gesetzt, was dann in der main den Übergang von Sleep zu Intro triggert. Die Logik für die Annahme und Auswertung der Responses ist hier auch schon festgelegt.	Funktioniert		
MotorFunktionen	Funktioniert ausgesprochen gut. Hier werden die Übungen für den Motor konstruiert. Alles, was man machen muss, ist, die jeweilige Steuerung zu referenzieren (also ohne Pause, mit Pause, asymmetrisch) und die entsprechenden Werte, die angefahren werden, wie schnell diese Werte angefahren werden sollen sowie die Pausen festzulegen. Einfach und unkompliziert. Man kann jede beliebige Übung erstellen, muss einfach eine der anderen Übungen kopieren und Werte und Namen anpassen. Man muss nur darauf achten, dass die Werte für den Motor anfahrbar sind.	Funktioniert		mehr Übungen?

navigation	Funktioniert ausgesprochen gut, limitiert nur durch die Tasten/Sensorik der Hardware. Ein Befehl kommt vom Board an, wird in MotorC interpretiert, über main, navigation weitergegeben an die derzeit aktivierte Auswahl/ui und entsprechendes Event wird getriggert. In der navigation werden die Items der jeweiligen uis eingeladen, damit diese entsprechend als Auswahl zur Verfügung stehen. In uis, die kein Knopf/Eingabefeld haben wie Intro, wird ein dummy Item erstellt, damit wir die Events kontrollieren können. In allen uis sind entsprechende Visualisierungen für das derzeit ausgewählte Item in der navigation. Das schließt einen Hover und einen Rahmen ein, der, wenn ein neues Item angepeilt wird, gezeichnet wird, sodass jede Auswahl entsprechend abgehoben wird.	Funktioniert		Statt Drucksensor in den Tasten ausgebaute Tastaturtasten als Eingabe verwenden. Alternativ links/rechts ausbauen und die linke Taste nur zur Eingabe verwenden und die rechte Taste zur Bewegung. Vermutlich etwas zuverlässiger.
pre_evaluation_ui	Identisch zur Evaluation, nur dass der Logger pre_evaluation zusammen mit Übungsauswahl und Auswahl der Dauer, Stresszustand in einer eigenen JSON speichert. Evaluation speichert dann nur die Auswahl des Stresszustands. Analysedatei baut aus diesen zwei JSON Dateien, die im Ordner logs abgelegt sind, die Statistik zusammen. Auswahl der Stressicons funktioniert einwandfrei und ist über die unterschiedlichen Icons intuitiv verständlich.	Funktioniert		
restart_ui	Funktioniert ohne Probleme. Nach Beenden der Übung wird nach Drücken auf Wiederholung wieder zurück in die Übungsauswahl geführt. Funktioniert einwandfrei. Einziger Punkt, der missverstanden werden könnte, ist das Wort "Wiederholen". Könnte dazu führen, dass die ausführende Person glaubt, es würde die bereits durchgeführte Übung wiederholen, statt eine neue Auswahl zu treffen.	Funktioniert		Audio und Wortwahl in JSON anpassen, damit klar ist, dass die ui ermöglicht, eine neue Auswahl der Übung zu treffen.

sleep	Spielt permanent die selbe Animation des schlafenden Neffy ab. Sobald der Sensorwert in MotorC registriert wird und das Callback in main gesetzt wird, triggert es das Aufwachen. Beim Aufwachen wird eine Animation abgespielt und der Motor fährt in die Ausgangsposition. Wenn die Person nicht den Intro Modus beendet und sich entfernt, wechselt es zurück in den Sleep Modus. Konsistent und zuverlässig. Der Thermalsensor reagiert manchmal anders als erwartet, aber es sollte kein Problem darstellen.	Funktioniert		Der Übergang zwischen den Modus Sleep zu Intro und Intro zu language_ui könnte an einigen Stellen flüssiger sein. Schnelleres Hochfahren der Motoren? Über gesamtes Programm einbauen, dass nach definierter Zeit ohne Aktion zurück in den Schlafmodus gewechselt wird
starting_pos_ui	Spielt Erklärungen für den User ab, kann übersprungen werden. Motorbewegung in Position 0 soll garantieren, dass Animation und Motorbewegung aus derselben Ausgangsposition starten und synchron laufen.	Funktioniert	Weil die zusätzliche Erklärung vor den Übungen nicht eingeplant war, kommt es dazu, dass wenn die Erklärung sofort übersprungen wird, der Roboter aber noch runterfährt, es so aussieht, als würde es in der starting_pos_ui steckenbleiben. Das ist ein etwas unschönes Verhalten und sieht für den User aus, als wären Motorbewegung und Animation nicht synchron.	Erst herrunterfahren bevor Erklärung abspielt wird und übersprungen werden kann? Am besten eine neue ui einfügen so das man Erklärung und Startposition sauber trennt.

SteuerungÜbung	Funktioniert, könnte etwas verbessert werden, wenn eine angepasste Kommunikation mit dem Board durchgeführt wird. Hier werden die Muster erstellt, mit denen die Übungen in MotorFunktionen referenziert werden. Es gibt eine Steuerung von Atemübungen ohne Pausen, mit Pausen und eine für asynchrone Übungen. Wenn eine Übung gestartet wird, dann wird über Logic der webp_sequence_player mit den definierten Animationen gestartet. Parallel wird der Motorcontroller und die Übungsauswahl an MotorFunktionen übergeben; diese übergibt dann MotorController und die definierten Werte der ausgewählten Übung an die jeweilige Steuerung. Wenn wir also handle_left (Referenz in Logic für die 4x6 Übung) aufrufen, wird der webp_sequence_player mit den definierten Animationen aufgerufen, die in Logic zusammengefügt werden. Dann wird die Übung in MotorFunktionen an die Steuerung ohne Pause übergeben und die Steuerung schickt die Befehle über MotorC an das Board. time.sleep wird verwendet, um die Zeiträume zu strukturieren. Mit dem Setzen des Batches wird die Motorbewegung und die abgespielte Animation synchronisiert. Ausgeführt im Bericht.	Funktioniert	Es kann sein das Informationen beim Transfer zwischen Board und Raspberry Pi nicht richtig interpretiert werden und es zu Fehlern im Programm kommt. Aus diesem Grund wird time.sleep verwendet anstatt den response vom Board das Motor in Zielposition angekommen ist. Das ist erheblich stabiler.	Warten auf die Response, dass der Motor in der Zielposition angekommen ist, bevor ein neuer Befehl geschickt wird, könnte eingebaut werden. Das Beispiel ist bereits in der Datei, funktioniert aber nicht einwandfrei, da wenn einer der Befehle zu früh oder spät ankommt oder nicht in der richtigen Reihenfolge, der Motor versucht, eine Position anzustreben, die bereits erreicht wurde. Heißt, der Motor spielt nicht mehr korrekt die Atemübung ab. Lösung wäre, statt zwei Befehlen für Kopf und Körper einen Befehl zu schicken, der sagt, ob beide Motoren ihr Ziel erreicht haben oder nicht.
ui	Auswahl der Übungen. Funktioniert wie gewünscht für die derzeitige Menge an Übungen. Alle Übungen sind nebeneinander sortiert. Es gibt einen Infotext, welcher über ein konstruiertes Modul, das einen Zeilenumbruch durchführt, die Informationen der Übungen im jeweiligen Panel darstellt. Einfach ergänz- und austauschbar. Für 5 Übungen noch okay, bei mehr problematisch, da Übungen nicht mit ausreichend Informationen dargestellt werden können. Problematik wird auch im Bericht ausgeführt.	Funktioniert	Mehr Übungen lassen sich in diesem Format nicht darstellen, da die notwendigen Informationen nicht mehr auf den Bildschirm passen.	Statt alle Übungen nebeneinander darzustellen, eine Listenstruktur verwenden, ähnlich der language_ui. Also eine Liste mit Übungsnamen und wenn eine Auswahl getroffen wird, zeigt die rechte Seite die Übungsinformationen an. Theoretisch möglich, unbegrenzt Übungen hinzuzufügen. Icons für individuelle Übungen anlegen?

volume_ui	Funktioniert gut. Ermöglicht es, die Lautstärke anzupassen; diese orientiert sich an einem voreingestellten Referenzwert auf dem Pi. Versuche, die Systemlautstärke des Pi über den Slider zu kontrollieren, waren nicht erfolgreich. Da Menschen nicht linear, sondern logarithmisch hören, wurde der Slider entsprechend angepasst. Im linken Bereich des Sliders wird die Lautstärke weniger stark verstärkt als im oberen. Grund dafür ist, dass Menschen bei leisen Geräuschen schon bei kleinen Veränderungen eine starke Veränderung wahrnehmen, wohingegen die Vergrößerung der Lautstärke im rechten Teil des Sliders größere Sprünge machen muss, da Menschen bei lauten Geräuschen Unterschiede wesentlich weniger stark wahrnehmen. Dafür verwendet wurde ein quadratisches Verhalten, was auf den Slider angewendet wurde. Wird im Bericht auch ausgeführt. Funktioniert jetzt wesentlich angepasster an menschliche Wahrnehmung.	Funktioniert		Man könnte noch komplexere Lautstärkekurven entwerfen
webp_sequence_player	Funktioniert wie beabsichtigt. Animationen werden in Batches abgespielt, die in logic aus den Animationen definiert werden. Batches werden synchron zur Motorbewegung gesetzt. Audios werden korrekt geloopt. Es können einfach Übungen über Anpassungen in logic, main und MotorFunktionen gestaltet werden.	Funktioniert		
System Atemerkennung				
nur Module die neu sind die anderen sind identisch oder sehr ähnlich zu System Standard				

MotorC	Funktioniert identisch wie System Standard. Mit dem Unterschied, dass jetzt die Befehle, die von dem Board über das Stresslevel Auskunft geben sollen, ausgelesen werden. Es wird außerdem ein Befehl an das Board geschickt, der dem Board sagt, dass die Stresserkennung beginnen soll.	Funktioniert		
Atemerkennung	Sagt dem User, was zu tun ist, damit die Atemerkennung durchgeführt werden kann. Kommunikation und Modul funktionieren zuverlässig; ob die Stresserkennung funktioniert, kann ich nicht beurteilen. Sobald in das Fenster der Atemerkennung geführt wird, wird der Befehl zur Ausführung der Stressidentifikation an das Board geschickt. Wenn der Befehl des jeweils identifizierten Stresslevels ankommt, wird eine zugeordnete Übung gestartet. Da es keine Daten gibt, welche Übungen bei welchem Stresslevel angewendet werden sollten, sind es beliebige Übungen mit beliebigen Zeiten.	Funktioniert	Wenn die Zeit auf dem Board abgelaufen ist und der User keine Eingabe gemacht hat, passiert nichts und das System muss neu gestartet werden.	Wissenschaftlich machen mit echten Daten. Durchführung der Stressidentifikation auf dem Board nicht an Zeit, sondern an Ein-Ausatmen binden.
main	Musste umgebaut werden; das gesamte System funktioniert. Auswahl Übung, Dauer, Restart nicht verfügbar.	Funktioniert		
System Profile				
evaluation_settings_ui	Erlaubt, Evaluation und Audiohinweise auszuschalten. Funktioniert wie gewünscht.	Funktioniert		
main_menu	Funktioniert. Man kann sofort auswählen, ob man ein Profil starten, die Auswahlen anpassen oder die Übung starten möchte.	Funktioniert		

profile_ui	Erlaubt, Profile über die Tastatur zu erstellen, zu löschen und zu laden. Funktioniert wie gewünscht. Auswahlen der Profile werden in der entsprechenden JSON gespeichert.	Funktioniert		Gibt keine Filter für die Namen. Man könnte die Tastatur verbessern. Zum Beispiel für unterschiedliche Sprachen aufbereiten, da Sprachen unterschiedliche viele Schriftzeichen haben und sich derzeit nicht alle darstellen lassen.
settings_manager	Wichtig für das Laden des derzeitigen Profils und seiner Auswahlen. Speichert Werte. Diese werden auch visuell in den Auswahlen geupdated.	Funktioniert		
settings_menu_ui	Steuert alle Auswahlen, also Dauer, Sprache, Evaluation Settings, Übung, Lautstärke. Merkt sich alle Auswahlen in diesen Auswahlen für das jeweilige Profil. Muss nach Erstellung eines Profils einmal konsequent durchgegangen werden.	Funktioniert		mehr Anpassungen? Audioauswahl?
main	Funktioniert wiederum ganz anders als die Mains der anderen Systeme. Kann in der Systemübersicht angesehen werden.	Funktioniert		
andere Module	Die anderen Module funktionieren etwas anders, da sie sich die Einstellungen merken und visuell updaten. Sie werden allerdings nicht alle noch einmal einzeln erklärt.			